

EC601

*advanced research,
real products, led by you*

Defining Your Project

Mission · Users · User Stories — from proposal to Sprint 1 backlog

EC601 · Running example: “Lab Radar” — an AI agent that watches arXiv for a research lab

Start with the Five W's (and How)

Journalism's oldest completeness check [\[ref\]](#) — if you can't answer all six in one line each, the project is not defined yet.

What? What are you building — one sentence, no jargon. *An agent that reads new arXiv papers and flags the few that matter to our lab.*

Who? Who uses it, and who else is affected? *PhD students preparing the weekly reading; the PI who trusts (or doesn't) the picks.*

Why? What breaks if it doesn't exist? *~150 relevant papers/week; students skim titles and miss the one that scoops them.*

When? When is it used — and when is it needed by? *Every Monday before lab meeting; demo-able by Sprint 3.*

Where? In what setting or workflow does it live? *Slack channel the lab already reads — not another dashboard nobody opens.*

How? How, at the architecture-sketch level? *arXiv API → relevance filter tuned on our papers → LLM evidence summary → Slack post.*

Exercise: write your six lines. The one you can't write is your first validation task.

Mission statement: one sentence, no escape

Adapted from Geoffrey Moore's positioning template (Crossing the Chasm) [\[ref\]](#) — every blank forces a decision:

For **[target user]** who **[has this need]**, the **[project]** is a **[category]** that **[key benefit]**. Unlike **[what they do today]**, it **[key difference]**.

Lab Radar, filled in:

For PhD students in our imaging lab who cannot read 150 new papers a week, Lab Radar is a monitoring agent that surfaces the 3–5 papers that affect our research, with evidence for each pick. Unlike keyword alerts, it explains why a paper matters and links the claim to the section that supports it.

The anti-example (reject this on sight):

“We use AI to revolutionize how researchers discover knowledge.” — no user, no need, no category, no alternative, nothing testable. If your mission fits any project, it defines no project.

Test: could a stranger reject your mission as false? If it cannot be wrong, it says nothing.

“Who” is plural — one user per subproduct

Most products serve several users, and “the user” hides that. Also separate users (touch it) from customers/deciders (adopt it) and stakeholders (affected by it) [\[ref\]](#). A clean way to manage multiple users: give each their own subproduct — own interface, own stories, own metric.

Lab Radar has three users — so three subproducts on one engine:

PhD student: the Monday digest — 5 picks in Slack, readable on a phone. *Success: never miss a relevant paper (recall).*

PI (also the decider): the audit view — every pick cites the passage that triggered it. *Success: spot-check a pick in 10 seconds (trust/precision).*

New lab member: the catch-up archive — searchable past digests by project. *Success: get oriented in a week, not a semester.*

Notice the conflict: the student’s metric is recall, the PI’s is precision — same engine, opposite tuning. Subproducts make that conflict visible instead of hidden in one averaged interface. Then make each user concrete in one paragraph: “reads on phone, 30 minutes on Sunday, forgives a boring pick but never a missed one.”

In 12 weeks you cannot serve everyone: pick the primary user, build their subproduct first, stub the rest.

User stories — the unit of work in this course

The Connextra template (2001), standard across agile teams [\[Cohn\]](#) [\[Atlassian\]](#):

As a **[specific user]**, I want **[capability]**, so that **[outcome I care about]**.

It is not a feature request — the “so that” carries the why, and the why is testable. Each story needs acceptance criteria: how we will know it’s done.

Quality check — INVEST [\[Wake\]](#): Independent · Negotiable · Valuable · Estimable · Small · Testable — detailed next slide.

Bad → good, same idea:

✗ “Add ML-based ranking to the pipeline.” (no user, no why, not testable)

✓ “As the PhD student, I want each pick to link the sentence that triggered it, so that I can judge relevance in 10 seconds without opening the PDF.” **Acceptance: 8/10 picks judged in ≤10s by a labmate.**

The user story is also your LLM prompt contract: hand the agent the story, hold it to the acceptance criteria.

INVEST — six checks, six failure modes

Bill Wake, 2003 [\[ref\]](#) — a review lens, not six boxes to tick; the letters trade off against each other.

I — Independent: No waiting on another story, so you can build in risk order. ✗ “Show feedback trends” before feedback collection exists.

N — Negotiable: A promise to talk, not a frozen spec — the how is decided when you build it. ✗ “Rank with cosine similarity over abstracts” — that’s implementation, not need.

V — Valuable: A vertical slice a user would notice — the check engineers break most. ✗ “Set up the database” — no user wakes up wanting a database.

E — Estimable: The team can size it. If not: too vague, or hides an unknown → run a spike (1-day time-boxed experiment; the deliverable is knowledge). ✗ “Filter by relevance” before anyone tested if relevance is detectable.

S — Small: Fits in one sprint. Too big = an epic → split it (next slide). ✗ Demo is “we’re 40% done.”

T — Testable: Done vs not-done is checkable by someone other than the author. ✓ “8/10 picks judged relevant by a labmate in ≤10s.”

Most common student violation: V — engineers decompose by architecture; stories decompose by user value.

“But login can’t fit in a sprint!”

The objection assumes the sprint unit is a complete, production-grade feature. It isn’t. Don’t build one layer at a time (database → UI → security — nothing works until everything works). Cut through all layers with a thin knife:

Slice 1: One hardcoded test user, plaintext check, ugly form — session works end-to-end. — *days* — *and it finds integration risk in week 1*

Slice 2: Real user table, hashed passwords, registration.

Slice 3: Password reset by email.

Slice 4: Rate limiting, logout — the hardening.

Slice 5: “Login with Google” (OAuth).

What genuinely resists slicing: security, compliance, crypto. Carry those as standing constraints on every slice (“always the auth library, never hand-rolled crypto”) — rules in your definition of done, not stories.

Nine named splitting patterns — by workflow step, by data variation, defer-quality, spike... [\[Lawrence, Humanizing Work\]](#) In this course you demo prototypes: Slice 1 is wrong to ship and exactly right to demo.

If you can’t make it small, make it thin: keep the end-to-end path, cut depth everywhere.

Lab Radar — the major stories (an epic, split)

Digest: As the PhD student, I want a Monday digest of ≤ 5 papers with one-line evidence each, so that I arrive at lab meeting prepared in 15 minutes. — *First demo — Sprint 1–2*

Trust: As the PI, I want each pick to cite the exact passage that made it relevant, so that I can spot-check without reading the paper. — *The make-or-break story*

Recall: As the lab, we want an alert within 24h if a paper overlaps our active projects, so that we are never scooped silently. — *Defines the metric: recall first*

Feedback: As the student, I want to mark picks useful/useless in one tap, so that next week's digest is better. — *Closes the loop — Sprint 3+*

Verify: As the course staff, we want the agent's summaries checked against sources, so that hallucinated claims never reach the lab. — *Your EC601 habit, productized*

Order = risk order: if “Trust” fails, nothing else matters. Build the story that can kill the project first.

Next steps — validate before you build

- 1. Rewrite your proposal as 3–5 assumptions.** Someone has this problem · we can build it with data we can access · it beats what the user does today.
- 2. Rank them: most fatal × most uncertain first.** Not the one you already know how to test — the one that kills the project if wrong.
- 3. Run the cheapest test that could prove you wrong.** Talk to 3 real users → mock it on paper → baseline on real data → only then build. If your first test is writing code, you skipped a cheaper one.
- 4. Write your kill criterion before testing.** “We pivot if...” — decided now, while you’re still honest.
- 5. Bring to next class:** mission sentence · named user · top-5 stories with acceptance criteria · assumption table with test results — in your GitHub repo.

Ask your LLM to list the five assumptions most likely to kill your project — the ones missing from your list are your blind spots.

Next steps

Milestones

- Wednesday 9/23/2026, complete form for [your team project proposal](#)
- Friday 9/25/2026, Osama will publish projects for your to vote on in case you want to change teams or join other teams
- Monday 9/28/2026, teams will be finalized

Thank you